A Reference Implementation and OMNeT++ Evaluation of a Low-Cost Hybrid ECC Authentication Protocol under Its Stated Environment

Dr. Shanta Sondur
Project Guide
Department of Information Technology
Vivekanand Education Society's
Institute of Technology
Mumbai, India
shanta.sondur@ves.ac.in

Jay Kerkar
Department of Information Technology
Vivekanand Education Society's
Institute of Technology
Mumbai, India
2023.jay.kerkar@ves.ac.in

Chaitanya Kelkar
Department of Information Technology
Vivekanand Education Society's
Institute of Technology
Mumbai, India
2023.chaitanya.kelkar@ves.ac.in

Chirag Poornamath
Department of Information Technology
Vivekanand Education Society's
Institute of Technology
Mumbai, India
2023.chirag.poornamath@ves.ac.in

Yash Kolekar
Department of Information Technology
Vivekanand Education Society's
Institute of Technology
Mumbai, India
d2024.yash.kolekar@ves.ac.in

***Abstract*—Al-Rasheed et al. proposed a hybrid elliptic-curve/symmetric-key mutual-authentication protocol for IoT devices and evaluated it in OMNeT++ without releasing code. We provide an open reference implementation of the protocol, a C port of its device side that runs on an ESP8266 microcontroller, and an OMNeT++ model of the environment the paper states (its Table II network and Table III per-operation times). Under those constants the model reproduces the paper's per-session processing cost of $2T_H+2T_{SE/D}$ per side (23.68 ms by the paper's arithmetic, 23.98 ms simulated) and the communication sizes of its Table IV for the nine schemes that have operation counts in its Table V. We then time the primitives on the ESP8266 and re-run the model with the measured constants, which lowers the per-session cost from 23.98 ms to 2.25 ms, and we study a question the paper leaves open: how a server resolves a pseudonym. With a hash-indexed lookup the per-session latency is independent of the number of devices from 100 to 1000; with a linear scan it grows with it. The implementation, the model, the 3,880-run sweep, the raw hardware logs and the scripts are open, and the model source runs unchanged on Linux and macOS.**

***Index Terms*—IoT authentication, elliptic curve cryptography, OMNeT++, ESP8266, reproducibility**

I. Introduction

Lightweight mutual authentication for resource-constrained devices is commonly evaluated by counting cryptographic operations and weighting them with per-operation times. Al-Rasheed et al. [1] propose a protocol in which devices and servers share long-term secrets from an offline phase and authenticate online with symmetric primitives only, and they evaluate it in OMNeT++ against other schemes. No implementation or simulation project accompanies the paper.

This paper contributes (i) an open reference implementation of the protocol (Python, AES-GCM, HKDF-SHA256, SHA-256) and a portable C device core that runs on an ESP8266 and interoperates with the Python server; (ii) an OMNeT++ model of the paper's stated environment, parameterised only by the paper's Tables II–V; (iii) a check that implementation and model reproduce the paper's per-session cost and communication sizes; (iv) hardware timings of the primitives behind the paper's cost constants, and the same model re-run with them; and (v) a scalability study of the server-side pseudonym lookup, which the paper does not specify.

II. Protocol and Implementation

We implement the nonce-based exchange of Section III of [1]. A trusted authority (TA) derives each entity's long-term secret $\lambda_X = H_1(ID_X \,\|\, s)$ locally and distributes pairwise keys $k^{\star}_{ij}$ in an offline phase. In an epoch t a device V_i derives a pseudonym $PID_{V_i,t} = H_2(\lambda_{V_i} \,\|\, t)$, samples a nonce N_i, and sends

$$M_1 = \langle PID_{V_i,t},\, C_1,\, \mu_1 \rangle,$$
$$(C_1, \mu_1) = \mathrm{AEAD}_{k^{\star}_{ij}}(PID_{V_i,t} \,\|\, N_i \,\|\, t).$$

The server resolves the pseudonym to a known device and epoch, verifies and decrypts M_1, samples N_j, answers with M_2 (the same construction), and both sides derive $K_{ij,t} = \mathrm{KDF}(k^{\star}_{ij} \,\|\, N_i \,\|\, N_j \,\|\, t)$. Replayed messages are rejected by an

epoch window per peer. We instantiate AEAD as AES-256-GCM [5], KDF as HKDF-SHA256 [4] and H as SHA-256, as suggested in the instantiation notes of [1]. The wire format is a length-prefixed JSON/base64 envelope.

The device side is also written as a portable C core on BearSSL [3] with the identical wire format. It runs natively against the unmodified Python server (12 of 12 handshakes) and on a NodeMCU ESP8266 over Wi-Fi (20 of 20 handshakes). In every handshake the session-key fingerprints on the two sides were identical and a replayed M_1 was rejected.

III. Evaluation Setup

A. Environment and constants

The OMNeT++ model uses the parameters of Table II of [1]: a 1500×1500 m area, 100 to 1000 devices, servers at 5% of the device count, a 10 Mbps channel, 75.6 mW transmit and receive power, a 15 μs time quantum, and a 500 m range. Operation times are those of its Table III ("reference values for the processing time"): $T_H = 0.32$ ms and $T_{SE/D} = 5.6$ ms. Operation counts and message sizes come from its Tables IV and V, and no parameter is fitted to any result. The model is written in plain OMNeT++ [2] 6.4.0 (no INET, no SUMO) and is our own implementation of the stated environment.

B. Assumptions the paper does not fix

The paper leaves several inputs open, and we state ours: an adversary share of 5% of the devices, each sending forged requests at a mean rate of 1 Hz; an authentication window of 10 s in which every device authenticates once at a uniform time; every server resolves every device in its range; and a first-come-first-served server CPU. Devices with no server in range cannot authenticate and are counted separately. Each result is the mean of five seeds. The assumption most relevant to the protocol is how a server resolves a pseudonym (Section IV-F).

C. Hardware testbed

Primitive timings were taken on a NodeMCU ESP8266 (Tensilica L106, stock 80 MHz and 160 MHz, about 80 KB RAM, no cryptographic hardware, Wi-Fi off) with BearSSL, timed by the cycle counter; each figure is the median of 100 runs (24 for elliptic-curve operations). Every benchmark output (hash, AES block, CTR, GCM ciphertext and tag, curve points) was checksummed, and all 35 digests are identical on both clocks and on a laptop running the same code. The whole benchmark was repeated on a second ESP8266 board: the 35 medians agree with the first board and with the earlier run to the microsecond.

IV. Results

A. Operation profile of the implementation

Table I lists what one handshake of the implementation executes (counted by instrumenting the primitives; the server was enrolled with 1, 20 and 100 devices, the target device last).

TABLE I
Operations and sizes per handshake. Table V of [1] lists $2T_H + 2T_{SE/D}$ per side.

	Device	Server
AEAD encrypt / decrypt ($T_{SE/D}$)	1 / 1	1 / 1
Pseudonym hash H_2	1	1 per candidate
Session-key KDF calls	2	2
Candidates scanned (linear)	—	about 8 per device
Message fields, M_1 / M_2	116 B / 57 B (1384 bits)	
On the wire (JSON/base64)	216 B / 133 B	

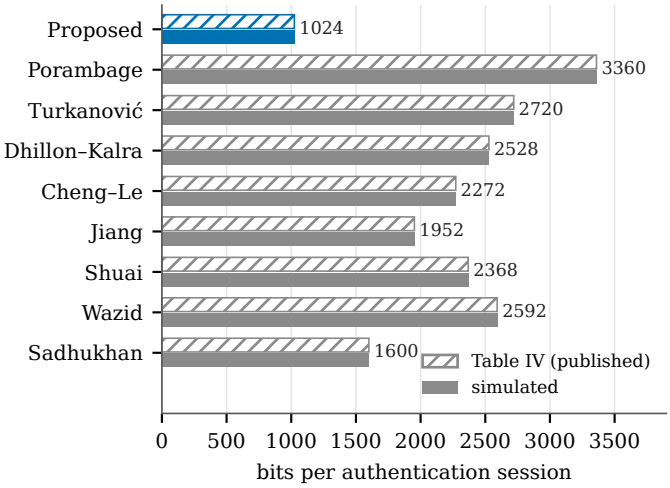


Fig. 1. Bits per authentication session: Table IV of [1] (hatched) and the simulation without adversaries (solid).

The symmetric operations match Table V. Table V does not list the session-key derivation required by the protocol's own key-derivation step or the pseudonym resolution; we report them separately so that the paper's accounting and the implementation's can be compared directly. Message size depends on the field encodings and the framing, which the paper does not fix; Table IV lists 1024 bits over four messages, and we simulate that figure and the 264-bit, two-message figure of its text.

B. Communication cost

Fig. 1 shows the bits per session in the simulation without adversaries for the nine schemes of Table IV that have operation counts in Table V. All equal the paper's values, and the proposed scheme has the lowest communication cost. Because Table IV's sizes are inputs to the model, this confirms the pipeline rather than the sizes themselves.

C. Per-session cost under the paper's constants

With Table III, Table V gives $2 \cdot 0.32 + 2 \cdot 5.6 = 11.84$ ms per side, 23.68 ms per session. Table II compares this with the simulation (no adversaries, hash-indexed lookup). The simulated latency is slightly above the arithmetic because it also contains airtime, rounding to the 15 μs time quantum and occasional queueing at a server. Adding the session-key

TABLE II
PER-SESSION PROCESSING COST (MS), $N = 100$, NO ADVERSARIES, HASH-INDEXED LOOKUP, MEAN OF FIVE SEEDS. "ARITHMETIC" IS TABLE V COUNTS × PER-OPERATION TIMES.

	Table III constants		ESP8266, 80 MHz	
Configuration	Arith.	Simulated	Arith.	Simulated
Table V as printed, 4 msgs	23.68	23.98	2.08	2.25
Table V as printed, 2 msgs	23.68	23.85	2.08	2.13
Table V + session KDF	—	35.18	3.34	3.51

TABLE III
MEASURED OPERATION TIMES ON THE ESP8266 (MS, MEDIAN OF 100 RUNS; 24 FOR CURVE OPERATIONS) AGAINST TABLE III OF [1]. AES-GCM: AES-256, 56 B INCLUDING KEY SETUP, FOR THREE BEARSSL AES IMPLEMENTATIONS. RATIO: 80 MHZ TIME OVER TABLE III.

Operation	Tab. III	80 MHz	160 MHz	ratio
T_H, SHA-1 (32 B)	0.32	0.0363	0.0182	0.11
SHA-256 (40 B)	—	0.0746	0.0373	—
$T_{SE/D}$, AES-GCM (fast)	5.6	0.485	0.244	0.09
small-table	5.6	0.834	0.419	0.15
constant-time	5.6	1.146	0.574	0.20
HKDF-SHA256	—	0.629	0.315	—
T_{ECM}, P-256	17.1	276.4	138.2	16.2
X25519	—	130.9	65.4	—

derivation, priced at a measured ratio to a hash since Table III has no KDF constant, gives 35.18 ms.

D. Hardware measurements

Table III and Fig. 2 give the measured cost of the primitives behind the three constants of Table III. Table III does not name the platform its values correspond to, so we report how they compare with one concrete constrained device. The ratios between operations differ from Table III's: on the ESP8266 a symmetric operation costs about 13 hashes ($T_{SE/D}/T_H$ for the fast AES) and a curve multiplication about 7,600, against 17.5 and 53 in Table III. The AES-GCM figure depends strongly on the AES implementation (0.48 ms table-based, 1.15 ms constant-time), which is a design choice a deployment must make.

End-to-end on the device. In the firmware, a handshake costs 1.67 ms to build M_1 and 3.59 ms to verify M_2 and derive the key, 5.26 ms of device compute in total (mean of 20 handshakes), and 52 ms end to end over Wi-Fi; M_1 is 216 bytes. The firmware uses the constant-time AES and includes JSON/base64 parsing, the per-message key schedule and the session KDF, so it exceeds the operation-count figure for the same constants (Fig. 3).

E. Simulation with measured constants

Re-running the model with the ESP8266 constants (Table II, right) puts the per-session cost of the proposed scheme at 2.25 ms with Table V's counts and 3.51 ms with the session KDF, against 23.98 ms and 35.18 ms with Table III. Communication cost is unchanged because it does not depend on

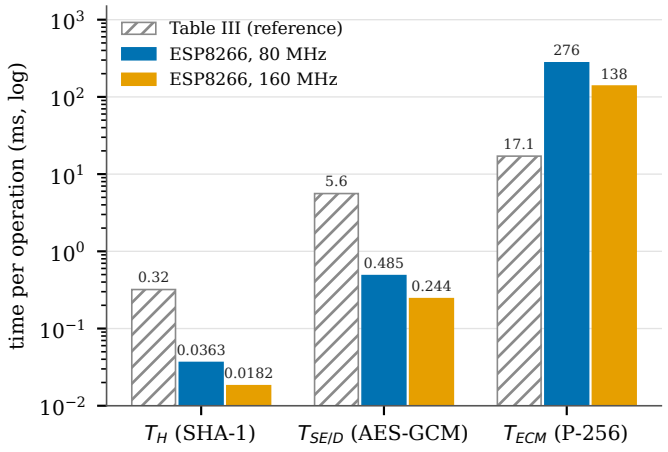


Fig. 2. Cost of the three primitives of Table III: the reference values of [1] (hatched) and the ESP8266 at 80 and 160 MHz (log scale). AES-GCM is the fast table implementation.

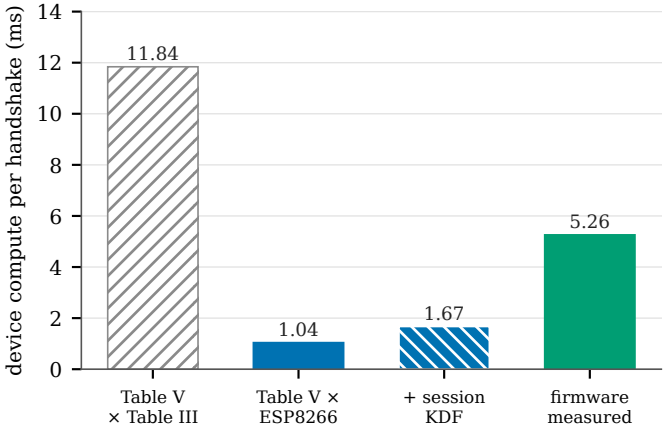


Fig. 3. Device-side compute per handshake. Hatched: Table V counts × Table III. Blue: the same counts × the measured ESP8266 constants (fast AES), without and with the session KDF. Green: measured end to end in the firmware (constant-time AES, parsing included).

processing times. The model structure is the same in both runs, and only the per-operation times differ.

F. Scalability and the pseudonym lookup

The paper does not say how a server maps an incoming pseudonym to a device. We simulate both natural choices: a linear scan over enrolled devices and epochs, as in our implementation, and a hash-indexed table, which costs storage proportional to the number of devices and epochs. Fig. 4 and Table IV show the mean per-session latency with the adversary share above, under both sets of constants.

With the hash index the latency is flat at the cost-model value. With a linear scan, latency grows with the number of devices a server must resolve, and the spread across seeds is large near saturation (standard deviation 2141 ms at 1000 devices with Table III constants, 370 ms with the ESP8266 constants). Faster primitives lower the curve by an order

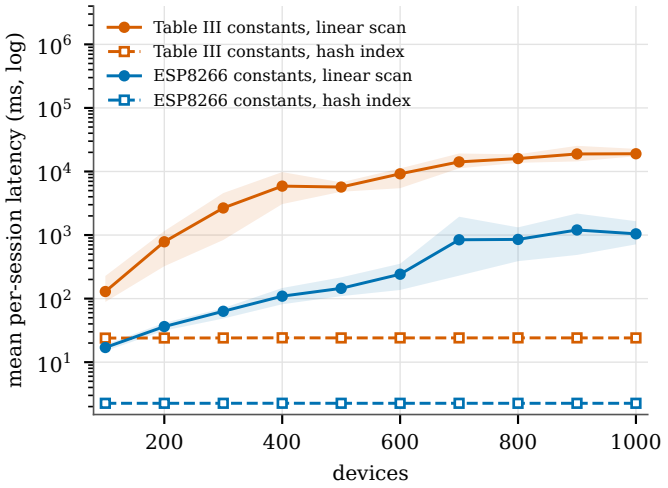


Fig. 4. Mean per-session latency of the proposed scheme against the number of devices (log scale; band: minimum to maximum over five seeds). Table V counts; adversary share 5%.

TABLE IV
MEAN PER-SESSION LATENCY (MS), MEAN OF FIVE SEEDS.

	Table III constants		ESP8266 constants	
Devices	Linear	Hash	Linear	Hash
100	129.2	23.93	17.0	2.25
300	2666.2	24.04	63.2	2.25
500	5691.2	24.07	145.0	2.25
700	14155.8	24.08	842.7	2.25
1000	19009.8	24.07	1044.6	2.25

of magnitude but do not change its shape. On a laptop-class server our Python implementation resolves a pseudonym in 4.8 ms at 1,000 devices and 23.9 ms at 5,000 (13-epoch window, worst case: device enrolled last); a precomputed table avoids the scan at the price of $N \times W \times 32$ B, 416 KB at 1,000 devices. The choice is therefore one a deployment must make explicitly, trading server time for storage.

G. Summary against the published values

Table V collects the quantities of [1] that can be derived from its stated tables and environment, and what we obtain.

H. Reproducibility

The model's 3,385-run main sweep was run on two toolchains (OMNeT++ 6.0.3 on Linux and 6.4.0 on macOS arm64); all rows agree to a relative difference of 5×10^{-14}. The 495 hash-index and uncontended runs were added on macOS only. The hardware benchmark is archived with raw serial logs for both boards and both clocks, and the C device core interoperates with the Python server on the same wire format. Scripts, configuration generators, result tables and the figure script are open.

V. LIMITATIONS

The model is ours, not the authors'; the assumptions of Section III-B are not specified by the paper and results at saturation depend on them. The results of Sections IV-C, IV-E and IV-F are model output, not measurements, and the ones under Table III constants use the paper's assumed values. Hardware timings come from one microcontroller family with software cryptography and one library; other devices, libraries or hardware AES will differ, and 8- and 16-bit parts were not measured. The end-to-end firmware figures are means over 20 handshakes recorded in our project repository; the per-primitive benchmarks, unlike them, are archived with raw logs. Constants without a measured counterpart (T_C, T_{fe}) are taken from [1]. We reproduce the quantities derivable from Tables II–V and do not evaluate the figures of [1], whose underlying data is not published; the other schemes of its tables are not re-implemented.

TABLE V
PUBLISHED VALUES AND OURS. THE FIRST TWO ROWS ARE CONSISTENCY CHECKS: THEIR INPUTS ARE THE PAPER'S OWN TABLES.

Quantity	Published	Ours
Bits per session, Table IV, nine schemes	1024 (proposed) to 3360	identical in all nine
Per-session processing, proposed, Table V × III	23.68 ms	23.98 ms (simulated); 23.85 ms (2 msgs)
Same counts, ESP8266 constants	not given	2.08 ms (arithmetic); 2.25 ms (simulated)
Device compute, firmware	not given	5.26 ms (20 runs)
Lookup, 1000 devices (Table III constants)	not given	24 ms (hash index); 19,010 ms (linear)

VI. CONCLUSION

We provide an open implementation, an ESP8266 port and an OMNeT++ model of the protocol of [1]. Under the paper's stated environment and constants, they reproduce its per-session processing cost and its communication sizes. Timing the primitives on a constrained device and re-running the model with the measured constants lowers the per-session cost by an order of magnitude, and the server-side pseudonym resolution, which the paper leaves open, determines how the protocol scales with the number of devices.

REFERENCES

[1] A. Al-Rasheed, R. Khan, A. Ahmad, A. Saeed, F. Alturise and S. Alkhalaf, "A low-cost hybrid elliptic curve cryptography authentication protocol for trustworthy Internet of Things communication," *IEEE Trans. Consum. Electron.*, vol. 72, no. 2, pp. 4483–4491, May 2026, doi:10.1109/TCE.2026.3662053.
[2] A. Varga and R. Hornig, "An overview of the OMNeT++ simulation environment," in *Proc. SIMUTools*, 2008.
[3] T. Pornin, "BearSSL, a smaller SSL/TLS library," [Online]. Available: https://bearssl.org
[4] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, 2010.
[5] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM)," NIST SP 800-38D, 2007.